

OOPS

1. Object-Oriented Programming (OOP)

OOP is a programming paradigm that organizes software using **objects**, which contain **data (attributes)** and **methods (functions)**. It improves code reusability, security, and maintainability.

2. Class

A **class** is a blueprint or template for creating objects. It defines the properties and behaviors of objects.

Example: Car is a class.

3. Object

An **object** is an instance of a class. It represents a real-world entity with its own data and behavior.

Example: MyCar is an object of the Car class.

4. Encapsulation

Encapsulation is the process of **wrapping data and methods into a single unit (class)** and restricting direct access to data using access modifiers.

Benefit: Data security.

5. Abstraction

Abstraction means **hiding implementation details** and showing only the essential features to the user.

Benefit: Reduces complexity.

6. Inheritance

Inheritance allows one class to **acquire the properties and methods of another class**.

Benefit: Code reusability.

7. Polymorphism

Polymorphism means "**many forms**." The same method can perform different tasks depending on the object.

Types:

- Compile-time polymorphism (Method Overloading)
- Run-time polymorphism (Method Overriding)

8. Constructor

A **constructor** is a special method that is automatically called when an object is created. It initializes the object's data.

9. Destructor

A **destructor** is a special method used to release resources when an object is destroyed (commonly used in C++).

10. Advantages of OOP

- Code reusability
- Data security
- Easy maintenance
- Modularity
- Scalability
- Easier debugging and testing

OPERATING SYSTEM

Operating System (OS) – Short Notes

1. Operating System (OS)

An **Operating System (OS)** is system software that acts as an **interface between the user and computer hardware**. It manages hardware resources and provides services for application programs.

Examples: Windows, Linux, macOS, Android.

2. Functions of Operating System

- Process Management
- Memory Management
- File Management
- Device Management
- Security and Protection
- User Interface (CLI/GUI)
- Resource Allocation

3. Process

A **process** is a program that is currently being executed.

States of a Process:

- New
 - Ready
 - Running
 - Waiting (Blocked)
 - Terminated
-

4. Thread

A **thread** is the smallest unit of CPU execution within a process.

Advantages:

- Faster execution
 - Better CPU utilization
 - Easy communication between threads
-

5. CPU Scheduling

CPU Scheduling decides **which process gets the CPU next**.

Scheduling Algorithms:

- FCFS (First Come First Serve)
 - SJF (Shortest Job First)
 - Round Robin (RR)
 - Priority Scheduling
-

6. Deadlock

A **deadlock** occurs when two or more processes wait indefinitely for resources held by each other.

Conditions for Deadlock

1. Mutual Exclusion
2. Hold and Wait
3. No Preemption

4. Circular Wait

7. Starvation

Starvation occurs when a process waits indefinitely because higher-priority processes keep getting CPU time.

Solution: Aging.

8. Thrashing

Thrashing occurs when the system spends more time swapping pages between RAM and disk than executing processes.

Cause: Insufficient memory or too many processes.

9. Memory Management

Memory management is the process of allocating and deallocating memory to programs.

Techniques:

- Paging
 - Segmentation
 - Virtual Memory
-

10. Paging

Paging divides:

- **Physical memory** into **frames**
- **Logical memory** into **pages**

It eliminates **external fragmentation**.

11. Segmentation

Segmentation divides memory into **variable-sized segments** such as code, data, and stack.

12. Virtual Memory

Virtual memory allows programs larger than RAM to execute by using secondary storage (disk) as an extension of main memory.

13. Page Replacement Algorithms

When memory is full, a page replacement algorithm selects a page to remove.

Algorithms:

- FIFO
 - LRU (Least Recently Used)
 - Optimal
-

14. Synchronization

Synchronization ensures that multiple processes or threads access shared resources safely and correctly.

Techniques:

- Mutex
 - Semaphore
 - Monitor
-

15. Semaphore

A **semaphore** is a synchronization mechanism used to control access to shared resources.

Types:

- Binary Semaphore
 - Counting Semaphore
-

16. Mutex

A **mutex (Mutual Exclusion)** allows only **one thread** to access a shared resource at a time.

17. Critical Section

The **critical section** is the part of a program where shared resources are accessed.

Requirements:

- Mutual Exclusion
 - Progress
 - Bounded Waiting
-

18. File System

A file system organizes and manages files stored on secondary storage.

Examples:

- NTFS
 - FAT32
 - ext4
-

19. Types of Operating Systems

- Batch Operating System
 - Multiprogramming Operating System
 - Multitasking Operating System
 - Time-Sharing Operating System
 - Distributed Operating System
 - Real-Time Operating System (RTOS)
-

20. Interrupt

An **interrupt** is a signal that temporarily stops the CPU to handle an important event.

Types:

- Hardware Interrupt
 - Software Interrupt
-

21. System Call

A **system call** is a request made by a user program to the operating system for services.

Examples:

- File operations

- Process creation
 - Memory allocation
-

22. Kernel

The **kernel** is the core part of the operating system that manages CPU, memory, devices, and processes.

23. Shell

A **shell** is a command interpreter that allows users to interact with the operating system.

Examples:

- Bash
- PowerShell
- Command Prompt (CMD)

Computer Networks

1. Computer Network

A **computer network** is a collection of interconnected computers and devices that communicate and share data and resources.

Examples: Internet, LAN in a college, office network.

2. Types of Networks

- **LAN (Local Area Network):** Covers a small area (home, office, school).
 - **MAN (Metropolitan Area Network):** Covers a city.
 - **WAN (Wide Area Network):** Covers large geographical areas (countries/world).
 - **PAN (Personal Area Network):** Covers a few meters (Bluetooth).
-

3. Network Topologies

- Bus

- **Star**
- **Ring**
- **Mesh**
- **Tree**

Most commonly used: Star Topology.

4. OSI Model (7 Layers)

Layer	Function
7. Application	Provides services to users
6. Presentation	Data encryption, compression
5. Session	Manages communication sessions
4. Transport	Reliable data delivery
3. Network	Routing, IP addressing
2. Data Link	Error detection, MAC addressing
1. Physical	Transmission of bits

Mnemonic (Top → Bottom):

All People Seem To Need Data Processing

5. TCP/IP Model

Layer	Protocols
Application	HTTP, FTP, SMTP, DNS
Transport	TCP, UDP
Internet	IP, ICMP
Network Access	Ethernet, Wi-Fi

6. IP Address

An **IP Address** uniquely identifies a device on a network.

Types:

- IPv4 (32-bit)
- IPv6 (128-bit)

Example: 192.168.1.1

7. MAC Address

A **MAC Address** is a unique **48-bit physical address** assigned to a network interface card (NIC).

Example: 00:1A:2B:3C:4D:5E

8. TCP (Transmission Control Protocol)

TCP is a **connection-oriented** protocol that provides reliable data transfer.

Features:

- Reliable
 - Error checking
 - Ordered delivery
-

9. UDP (User Datagram Protocol)

UDP is a **connectionless** protocol used for fast communication.

Features:

- Faster than TCP
- No guarantee of delivery
- Low overhead

Applications: Video streaming, online gaming, VoIP.

10. TCP vs UDP

TCP	UDP
Connection-oriented	Connectionless
Reliable	Unreliable
Slower	Faster

TCP

Error checking

Used for Web, Email

UDP

Minimal error checking

Used for Streaming, Gaming

11. DNS (Domain Name System)

DNS converts **domain names** into **IP addresses**.

Example: google.com → IP Address

12. HTTP & HTTPS

- **HTTP:** Transfers web pages; not encrypted.
 - **HTTPS:** Secure version of HTTP using SSL/TLS encryption.
-

13. FTP (File Transfer Protocol)

FTP is used to transfer files between computers over a network.

14. SMTP, POP3, IMAP

- **SMTP:** Sends emails.
 - **POP3:** Downloads emails to one device.
 - **IMAP:** Synchronizes emails across multiple devices.
-

15. Router

A **router** connects different networks and forwards data packets based on IP addresses.

16. Switch

A **switch** connects devices within the same LAN and forwards data using MAC addresses.

17. Hub

A **hub** broadcasts data to all connected devices without filtering.

18. Modem

A **modem** converts digital signals to analog and vice versa for Internet communication.

19. Gateway

A **gateway** connects networks using different communication protocols.

20. Firewall

A **firewall** protects a network by monitoring and filtering incoming and outgoing traffic.

21. Bandwidth

Bandwidth is the **maximum amount of data** that can be transmitted over a network in one second.

Unit: bps (bits per second)

22. Latency

Latency is the **time taken** for a data packet to travel from source to destination.

Unit: milliseconds (ms)

23. Packet

A **packet** is the basic unit of data transmitted over a network.

24. Packet Switching

Packet switching breaks data into small packets, sends them independently, and reassembles them at the destination.

25. Unicast, Broadcast, Multicast

- **Unicast:** One sender → One receiver.
 - **Broadcast:** One sender → All devices in the network.
 - **Multicast:** One sender → Selected group of receivers.
-

26. Network Security

Common security mechanisms include:

- Firewall
- Encryption
- VPN
- Antivirus
- Authentication